Aurora

Project context: how the whole thing works

A local-first music player for the songs already on your devices,
with a beat-matching AI DJ, mood journeys and an immersive Now Playing.

Version 1.8.0 (build 21) · October 2026
Repository: github.com/Kailu687/Aurora · Site: kailashamarnath.me/Aurora

Written for people and for AI assistants who need to understand,
change or extend Aurora without reading every file first.

0. Read this first

This document is the single place to learn what Aurora is, how its parts fit together, where each behaviour lives in the code, and the rules the project follows. Section numbers are stable so they can be referenced. If you are an AI assistant: the "Rules" in section 15 are the owner's standing instructions and override generic habits.

Question	Short answer
What is it?	A music player for local files (no streaming service). One web code base runs as a website/PWA, an Android app and a Windows app.
Frontend?	Plain HTML, CSS and JavaScript in **web/**. No framework, no bundler, no build step for the app itself. One global object **A**.
Backend?	**None.** GitHub Pages hosts the website, the web app and the update feed as static files. Everything else runs on the device.
Native shells	Android: Capacitor 8 plus four Java classes. Windows: Electron 33 (main.js + preload.js).
How do users get new versions?	Web code updates over the air from a signed **update-manifest.json**; native changes need a new APK or installer.
Where is user data?	Only on the device: IndexedDB (songs index, covers, lyrics, backups) and localStorage (playlists, history, settings). Never uploaded.
Paid services?	None required. The only optional paid thing is an Anthropic API key for DJ chat lines; every other feature is free and local, including the studio voice (an open neural voice that runs on the device).
Owner	Kailash Amarnath (GitHub Kailu687). Releases are confirmed by him before they go out.

Related project: Aurora Pocket

Aurora Pocket is a separate hardware project in the same family: a wired-only pocket music player (its design files live outside this repository). Its product page is part of this repository's website at **tools/site/AuroraPocket/**. The Pulse visual engine in the app (section 10) was written so it could later drive a display on Aurora Pocket.

1. What the user gets

Forms of the app

Form	What it is	Where
Web app (PWA)	Website that installs like an app, works offline	kailashamarnath.me/Aurora/app/
Android app	Web app in Capacitor + native folder access, media notification, text-to-speech	Aurora.apk on the Downloads page
Windows app	Web app in Electron + local file server, folder scan, mini window, global keys	Aurora-Setup-x.y.z.exe on GitHub Releases
Website	Marketing pages, downloads with checksums, release notes	kailashamarnath.me/Aurora/ (source: tools/site/)

Feature map (1.7.0)

Area	Features
Library	Link a folder (songs are never copied), instant listing then background tag reading, covers from tags / cover.jpg / one online lookup, albums, artists, genres, search, liked songs, playlists, **smart playlists** (rules that update themselves), edit song info.
Playback	Two-deck engine with crossfade, 10-band EQ, normalisation, speed, sleep timer, A-B loop, resume where you left off, lock-screen / notification controls, media keys.
Home	**Just press play** (one button: a song for this hour, then continuous play), **Today's picks** (rotates daily: forgotten favourites, never played, mood of the hour), memory card (On This Day), Made for you mixes incl. Forgotten Favourites.

Area	Features
Perfect Next Song	When the queue runs out Aurora keeps choosing one fitting song at a time: **Familiar**, **Surprise me** or **Somewhere new**. Learns from quick skips, lets energy drift gently, shows why each song was picked.
Flow Architect	Build a journey between moods (e.g. Calm → Nostalgic → Energetic → Euphoric) for 20 min to 2 h. Preview curve (planned vs real energy), flags abrupt changes, drag to reorder, swap or remove songs, play or save as playlist.
AI DJ	12 vibes, journeys (steady / peak / build / wind down), short mood-matched talk (optional on-device studio voice), live requests and nudges. **Real mixing**: beat grid, chorus (hook) detection, key detection, tempo matching, beat lock, mood-based transitions (bass swap, filter blend, reverb wash, echo out, turntable brake, build-up and drop), "Best parts" mode.
Lyrics	Synced lines that fill as they are sung, interludes, tap to seek, timing nudge, full-screen lyrics, **Saved lines** collection with share image, **Find lyrics** from LRCLIB (free, no key). Plain lyrics shown honestly as unsynced.
AI Audition	A feature page (not onboarding). Auto mode picks 3 library songs (low/mid/high melodies, prefers ones with lyrics); manual mode lets the user pick 1 to 5. The user sings along with lyrics while the music plays quietly. On-device YIN pitch tracking gives sung range (p5/p95), comfort zone (p25/p75), steadiness, brightness, breathiness and power. Songs are ranked by how their analysed melody range (mix.mel histogram) fits the comfort zone, with octave and +/-5 semitone suggestions. Honest by design: no accuracy scores, no "you sound like X", recordings never leave the device. Android needs app build 21+ (RECORD_AUDIO).
Immersion	Now Playing opens on play, colours from the cover, **Pulse** visuals that follow the sound (Aurora, Ambient, Bloom, Spectrum, Still), zen mode fades controls, songs hand over with an art transition.
Anywhere	Mini player that floats over other windows (Windows window / Chrome-Edge picture-in-picture), editable keyboard shortcuts, global shortcuts on Windows.
Stats	Listening stats and a story-style **Recap** (last 30 days, this year, all time) with a shareable image.
Safety	Signed updates, staged install, automatic rollback, daily + pre-update backups, merge-only restore, export/import.

2. Repository map

```
Aurora/
+- web/                     THE APP (shared by all platforms)
|  +- index.html            skeleton, CSP, script load order
|  +- css/style.css         all styling (glass look, phone layout, themes)
|  +- js/*.js               the program, one file per concern (section 3)
|  +- sw.js                 service worker: offline shell, staged/live update caches, boot guard
|  +- changelog.json        release notes shown in "What's new"
|  +- config.json           updateUrl + updateUrlFallback
|  `- manifest.webmanifest, icons/
+- android/                 Capacitor project; java/app/aurora/player/*.java
+- electron/                main.js (window, app:// server, IPC), preload.js (window.desktop),
|                           update-key.txt (public update key)
+- tools/                   publish-web.js (bump, sign, build site), serve.js, hash-releases.js,
|                           build-android.ps1, build-all.ps1, make-icons.js, make-test-music.js, site/
+- downloads/               Aurora.apk + hashes.json served by the website
+- docs/                    PROJECT.md, RELEASING.md, changelog.json, this PDF
+- .github/workflows/pages.yml   builds + signs + deploys the site on every push to main
+- capacitor.config.json, package.json (scripts, electron-builder config)
NOT in git: node_modules, build output, Android keystore, update signing key (*.pem)
```

3. The web app, module by module

Scripts load in this exact order from **index.html**; each adds to the global **A**. Later files may use earlier ones at load time; anything else is only used at run time. There are no imports, no npm packages in the browser and no build step.

File	Owns	Key exports
version.js	Version stamp written by npm run bump	A.VERSION {name, build, notes}

File	Owns	Key exports
util.js	DOM helpers, formatting, hash, icons, toasts, settings store	A.$, A.esc, A.icon, A.store, A.toast
db.js	IndexedDB wrapper (database aurora-db)	A.db.get/put/del/keys/getAll
safety.js	Backups, snapshots, mirror of playlists/history/saved lines, secrets	A.safety
meta.js	Tag reading (ID3/MP4), covers, audio analysis, **beat grid**, LRC parsing	A.analyze, A.mixAnalysis, A.parseLRC
engine.js	Audio graph: two decks, EQ, ducking, analyser, per-deck high-pass + bass filters, crossfade and **mixTo**	A.engine (E)
library.js	Scanning, enrich, covers, derived lists, playlists, smart playlists, mixes, lyrics storage	A.lib (L)
player.js	Queue, order, shuffle/repeat, history, resume, sleep timer, A-B loop, media session	A.player (P), EventTarget
dj.js	Vibes, scoring, similarity, background analysis queue, DJ sets, journeys, what the DJ says and when	A.dj, A.VIBES, A.analysis, A.speech
voice.js	Studio voice: Kokoro-82M neural TTS on the device (kokoro-js in web/vendor/kokoro), download, cache, render, play	A.voice
mixer.js	DJ transitions: mood-based style choice, plan, effects, fire, beat lock, mix strip UI	A.mixer, A.MIX_STYLES
updater.js	Signed OTA updates, staging, What's new, native-app notice	A.updater
ui.js	Cover colours, modals, menus, rows/cards, edit dialogs	A.ui (U)
features.js	Smart playlist editor, journey curves, memory card, Recap	A.feat
views.js	Every page and the router	A.render, A.routes, A.actions
flow.js	Moods, Flow Architect, Perfect Next Song, Just press play, Today's picks	A.flow, A.MOODS
lyrics.js	Lyric rendering/sync, Find lyrics (LRCLIB), saved lines, full-screen	A.lyr
audition.js	AI Audition: mic capture, YIN pitch, voice profile, song fit ranking, auto/manual modes, share card	A.audition (AU)
shell.js	Player bar, Now Playing, side panels, click/key handling, **boot()**	A.openNp
immersive.js	Pulse visuals, zen, song hand-over, auto-open	A.immersive
anywhere.js	Mini player, keymap + editor, global shortcuts	A.anywhere
native.js	Android bridge glue (notification, folders, TTS, HTTP)	A.native

How the pieces talk

- **Clicks**: any element with **data-a="name"** runs **A.actions[name](el, event)** (one delegated listener in shell.js). New modules add actions with Object.assign(A.actions, {...}).
- **Pages**: the hash is the route (#/flow, #/lines, #/album/<id>). **A.routes[name](el, arg)** renders into #view; A.render(true) re-renders softly.
- **Events**: the player is an EventTarget: **track** (new song), **state** (play/pause), **queue**. The engine emits **tick** and **vol**. The library has L.on(fn) for like/changed. DJ, analysis, mixer and flow each keep a listeners Set.
- **Boot** (shell.js boot()): install a staged update if any, wait for DOMContentLoaded, load the library, recover mirrored data, render, restore the session, start background jobs (tags, covers, analysis, update checks, daily snapshot).

4. Data model and storage

IndexedDB (aurora-db)

Store	Key	Contents
tracks	id	One record per song (fields below)
art	album key	Cover thumbnails (JPEG blobs, about 400 px)
roots	id	Linked folders (handle, path or Android tree URI)
files	id	Only for browsers that cannot link folders (copied audio)
kv	string	**lrc:<trackId>** lyrics you added or found · **snap:<time>** backups (last 6) · **mirror:playlists / history / savedLines** safety copies

Track record

id, title, artist, albumArtist, album, genre, year, duration, kind (handle | path | saf | blob), location, artKey, plays, last, liked, edited, meta (tags read), coverTried, lrc (sidecar file), and **an** (analysis):

```
an = { energy 0-1, tempo bpm, bright 0-1, pulse, loud, rms, dance 0-1, valence 0-1 (sad..happy), key (Camelot, e.g.
       mix: { v: MIX_VERSION (3), bpm (exact), beat0 (s), bar0 (first downbeat, s), bars: [energy per bar 0-100],
              hb / he (hook = repeated chorus, first and end bar), ib (bar to bring the song in), lb (end of last
              full-energy bar), key { name, cam, minor, clar }, dance, vocal, dur,
              hook / i0 / last (older 8-bar phrase fields, kept for compatibility) } }
```

localStorage (all keys start with aurora:)

Library: playlists, history, session, songSort. Playback: volume, shuffle, repeat, crossfade, normalize, eq, directAudio. Look: theme, accent, blur, bgAnim. DJ: djEnabled, djLast, djStyle, djTalk, djVoice, djVoiceOn, djJourney, djMix, studioVoice, studioVoiceName, studioVoiceReady, aiKey (moves to encrypted storage on Windows), aiModel, analyze. Flow / next song: flowSel, flowDraft, contMode, autoplay. Lyrics: savedLines, lrcOffsets, autoLyrics, lyrMiss. Immersion: immersive, zen, pulseStyle. Anywhere: keymap, globalKeys, globalOn. Updates: updateMode, autoUpdate, updateUrl, seenVersion, skipBuild, liveBuild, nativeSeen, lastSnapshot.

A backup (safety.collect) contains playlists (with song ids and artist|title|duration signatures so they survive a re-scan), likes, play counts, edits, history, typed lyrics and all aurora: settings except secrets and session data.

5. From a file to your ears

Platform	Audio URL	Served by
Android	https://localhost/_media/<doc id>	MediaWebViewClient streams from the SAF folder with Range support
Windows	app://local/__lib/<path>	electron/main.js, only for paths inside allowed folders
Chrome / Edge	blob: URL from the folder handle	the browser (file stays where it is)
Other browsers	blob from IndexedDB files	copied fallback

Engine graph (Web Audio): each deck is <audio> → source → high-pass → low-pass → low-shelf (bass) → deck gain → 10-band EQ, plus pre-fader sends into a tempo-synced echo and a reverb, and a noise riser (DJ effects) → normaliser → duck → master → analyser → speakers. On Android the engine runs in **direct mode** (plain <audio>, no graph) because the Android web view can go silent through Web Audio; EQ and the live analyser are unavailable there, and visuals fall back to the beat grid.

6. The music intelligence (all on the device)

Analysis (meta.js)

- **A.analyze** decodes a slice of the song and measures energy, tempo, brightness, groove and loudness.
- **A.mixAnalysis** builds the beat grid: onset envelope → autocorrelation for the tempo range → comb filter for exact BPM and phase → downbeat (bar0) from where chords change and the bass hits → energy per bar. Then a chroma (12-note) profile per bar finds what **repeats**: the hook is the loudest stretch that also comes back through the song (a chorus), extended to its full length (8 to 24 bars) so a mix never cuts it off. The overall

chroma gives the key and major/minor (Krumhansl profiles, shown as Camelot). Files over 14 MB skip the grid (tracked with anMixFail). Takes about 1-1.5 s per song more than 1.7.

- Mood features: **valence** (major/minor with key clarity, brightness, tempo, groove) and **dance** (how strongly onsets land on the beat). Vibes have val and dance ranges, so Sad prefers minor, low-valence songs and Party prefers danceable ones.
- The background queue in dj.js (A.analysis) analyses songs while the app is idle and re-analyses songs whose grid is from an older MIX_VERSION.

AI DJ (dj.js) and the mixer (mixer.js)

- Song choice: vibe score (genre/keywords, energy fit, tempo fit, era) + journey target energy + smooth step from the last song + bonus when tempos can be beat-matched (within 6%, half/double time allowed).
- Mix modes: **Best parts** (bring each song in on the build into its hook, play the whole hook, mix out right after it), **Full songs** (blend through outros), **Simple fades**.
- Transition style is picked by the mood of the set (MOOD_MIX in mixer.js): Hype/Workout favour echo outs, build-ups and brakes; Party/Feel Good bass swaps; Chill/Late Night/Focus filter blends; Sad/Wind Down reverb washes. Clashing keys or tempos too far apart avoid long overlaps. Blend styles (swap, filter, wash) overlap beat-locked songs for 4-16 bars; cut styles run an effect on the old song first (E.preFx: echo, brake, riser) and drop the new song in on the downbeat, straight onto its hook.
- Song choice also rewards compatible keys (A.keyMatch on the Camelot wheel).
- Firing: a timer fires exactly on the planned beat; **E.mixTo** starts the second deck already filtered. A 100 ms loop measures beat phase between decks, snaps once, then nudges the rate (phase error settles under ~0.01 beat, about 5 ms), and finally glides back to normal speed.
- Skip during a set = mix out on the next bar instead of a hard cut.
- Talking (dj.js): lines come from mood banks (high / warm / mellow / soft), are short, use the title only sometimes and never read tempo or full credits. dj.prepare() writes and renders the next line while the current song plays; the mixer leaves quiet bars for it (talkRoom) and the DJ speaks after the blend and before the hook, with the music dipped to about 40%. Over a build-up, a hype DJ shouts a short line timed to end on the drop.
- Studio voice (voice.js): Kokoro-82M, q8, WebAssembly. The engine JS ships in web/vendor/kokoro; its 21 MB wasm is downloaded from the site's /app/voice/ and checked against a pinned SHA-256; the 90 MB model comes from Hugging Face once and is cached. Falls back to system / phone text-to-speech.
- Optional: with the user's own Anthropic key, free-text requests and spoken lines can be written by Claude. Everything else works without it.

Moods, Flow Architect and Perfect Next Song (flow.js)

- **Moods** (A.MOODS): Calm, Dreamy, Melancholy, Nostalgic, Warm, Focused, Upbeat, Energetic, Euphoric, Intense. Each has target energy, tempo, brightness, colours and genre hints; Nostalgic also rewards older years and songs not heard for months.
- **jump(a, b)** measures how jarring a transition is (energy, tempo with half/double folding, brightness). Used everywhere to avoid abrupt changes.
- **Flow build**: walk through the requested length; at each point blend the two neighbouring moods (smoothstep), pick the song with the best mood fit minus transition cost, avoid repeating artists, stop near the target length. Swap uses the same score with both neighbours.
- **Perfect Next Song**: pickNext scores every song against the current one by mode (Familiar: similarity, likes, plays; Surprise me: 28% curveballs from another genre with the same feel; Somewhere new: unplayed / forgotten songs with a smooth feel), penalises recent plays and anything like a song skipped within 30 s, and

lets target energy drift slowly (up in the day, down late at night). refreshTail always keeps one chosen song queued so crossfades and mixes can see it coming; reasons show in the queue.

7. Lyrics (lyrics.js)

- Sources in order: lyrics stored in kv (typed or found), a .lrc file next to the song, lyrics inside the tags.
- **Find lyrics**: LRCLIB (lrclib.net, free, open, no account). First /api/get with title, artist, album, duration; then /api/search; the closest length wins and synced beats plain. Only those four fields are sent. Results are saved, misses are remembered for a week. Automatic lookup happens only when the lyrics panel is open and can be turned off.
- Synced view: current line lit and filled by progress, distance-based fade, "• • •" for gaps longer than 14 s, tap to seek, per-song timing offset (±0.5 s steps). Plain lyrics are labelled as not time-synced.
- Saved lines: heart on any line; page #/lines; play from that line; 1080×1350 share image.

8. Immersion and Aurora Anywhere

- **Pulse** (immersive.js): a canvas behind Now Playing. Each frame it reads 32 analyser bands (bass, mids, highs, level) and detects kicks from bass flux; without live data (Android) it synthesises the pulse from the beat grid and per-bar energy. Colours come from the cover accent variables. Soft styles draw at 30% resolution and are blurred by CSS for cheap glow. Runs only while Now Playing is open and the page is visible.
- **Zen**: after 4 s without input while playing, controls fade; any movement brings them back. **Hand-over**: the old background fades under the new one and the new cover settles in.
- **Mini player** (anywhere.js): on Windows the same window shrinks to 440×150, always on top, bottom-right (IPC set-mini), so playback never restarts. In Chrome/Edge a Document Picture-in-Picture window hosts the same element. Shows art, title, current lyric line, seek bar and controls.
- **Shortcuts**: A.KEY_ACTIONS defines actions and defaults; users rebind in Settings (one key per action). On Windows, global shortcuts (default Ctrl+Alt+Space / → / ← / M / L) are registered by main.js (IPC set-global-keys) and arrive as global-key events. Media keys always work.

9. Updates, signing and data safety

```
push to main -> GitHub Actions: node tools/publish-web.js --out site  (needs secret AURORA_UPDATE_KEY)
             `-> site/app/update-manifest.json = { name, build, notes, published,
                    files:[{path,size,sha256}], native:{android:{build}, windows:{version}}, sig }
installed app -> fetch manifest -> verify ECDSA P-256 signature -> build newer?
             -> snapshot user data -> download every file, check size + SHA-256 -> STAGE
             -> next start (or "Restart now") promotes staged -> live
             -> boot guard: 3 failed starts -> drop live update, back to bundled version
```

- **Signed payload** (identical in publish-web.js, updater.js and electron/main.js): JSON.stringify({name, build, notes, published, files:[{path,size,sha256}], native}). Signature: base64 IEEE-P1363. Public key (SPKI base64) is stamped into web/js/updater.js and electron/update-key.txt by publish-web.js --new-key; the private key lives at D:\GitHub\Aurora-keys\update-signing-key.pem and in the GitHub secret, never in git. The site build refuses to run without the key or if it does not match the stamped public key.
- Browser/Android: caches aurora-staged → aurora-live, served first by sw.js. Windows: userData/web-staged → web-live. A newer bundled build (new APK/installer) discards an older downloaded update.
- **Update modes**: Automatic (download in background, install on next start) or Ask me first. A small notice appears when a newer APK/installer exists (native changes cannot come over the air).
- **Data is never part of an update.** Snapshots: daily and before every update (last 6 in the app; on Windows also %APPDATA%\Aurora\backups, last 10). Playlists, history and saved lines are mirrored into IndexedDB and restored if localStorage is ever cleared. Restore merges: likes and counts only go up, playlists matched by id then name, nothing deleted. The Windows uninstaller keeps app data.

- Proven in testing: a 1.6.2 client updated itself to 1.7.0 with likes, playlists and the API key intact; tampered manifests (changed notes, hash, removed signature, bumped build) are refused; a broken update rolls back.

10. Native shells

Windows (electron/)

Window with hidden title bar, sandboxed renderer, contextIsolation, no node integration. app:// protocol serves the web app (from web-live if a valid update is active) and audio only from folders the user picked (allow-list in userData/folders.json; dropped files are allowed individually). IPC is only accepted from app:// pages.

IPC (window.desktop): pickFolder, scan, adoptRoots, fetchJSON (HTTPS only), downloadUpdate, applyUpdate, bootOk, shellInfo, secretGet / secretSet (Windows encryption via safeStorage), saveSnapshot, relaunch, pathFor, onMediaKey, setMini, setGlobalKeys, onGlobalKey.

Android (android/)

Capacitor serves web/ at https://localhost. AuroraBridge (window.AuroraAndroid): appBuild, pickFolder, scan, hasAccess, status, requestPermission, openSettings, update (notification), speak / stopSpeak (TTS), httpGet (covers; Apple hosts only), stop. PlayerService = media notification + background keep-alive; closing from the app switcher stops everything. MediaWebViewClient streams songs from the chosen folder.

11. Website and publishing

URL	Source	Notes
/Aurora/	tools/site/index.html	Features, platforms, open-source and security section
/Aurora/downloads/	tools/site/downloads/	Reads GitHub Releases live, falls back to releases.json
/Aurora/AuroraPocket/	tools/site/AuroraPocket/	Aurora Pocket hardware page
/Aurora/app/	web/	The web app + update-manifest.json (OTA feed)

A GitHub Release does not update installed apps by itself. Installed apps follow update-manifest.json, which is rebuilt on every push to main. Releases carry the APK/installer and feed the Downloads page.

```
npm run web                      dev server on :5173
npm start                        desktop app from source
npm run bump -- --notes "a|b" [--name 1.7.1]   version + build + changelog everywhere
npm run android | windows | release | hash      native packages + checksums
node tools/publish-web.js --new-key             create the update signing key (once)
```

You changed	Users get it by
anything in web/	Over the air (after merge to main)
android/ Java, permissions, icons	New APK (install over the old one, same signing key)
electron/main.js, preload.js	New Windows installer

12. Privacy and security model

- No accounts, analytics, ads or server of our own. Open source (MIT).
- Network use, all listed: update feed (GitHub Pages), cover lookup (Apple public search, artist/album only, can be off), lyrics lookup (LRCLIB, title/artist/album/length, can be off), Downloads page reads GitHub Releases, optional Anthropic API with the user's own key.
- Content Security Policy: scripts only from the app itself; no inline handlers.
- Updates are signed and hash-checked; Electron is sandboxed with a folder allow-list; secrets encrypted on Windows.
- Not audited by a third party. Windows installer not code-signed yet (SmartScreen may warn).

13. Testing approach

- No unit-test framework. Behaviour is verified by driving the real app with Playwright (Chromium) and Electron (Playwright _electron under xvfb), with generated test songs (tools/make-test-music.js, plus synthetic beat-accurate songs for the DJ).
- Checks added for 1.8: hook finder on synthetic songs with real chord progressions (loud bridge, 2-bar pickup, hook-first) all exact; every transition style driven in Chromium with level and filter-reset logs; DJ talk timing logged against the mix; Electron loads the voice engine under the CSP.
- Checks used for 1.7: UI screenshots of every new screen at desktop and phone sizes with zero console errors; DJ transitions on phrase boundaries with rate and phase logs; Flow build, drag reorder and swap; LRCLIB responses mocked; Electron mini window, always-on-top and global keys; old-client-to-new update with data intact; signature tamper tests; boot-guard rollback.

14. Known limits and open items

- The studio voice was built and wired up in a sandbox that could not reach Hugging Face; the engine, wasm download, checksum and CSP were verified there, and full speech is verified on a real PC.
- The studio voice renders on the CPU (WebAssembly): fast on PCs, slower on older phones. Lines are rendered while the previous song plays, so this only delays the very first line.
- The hook finder is tuned on synthetic songs; real songs with unusual structure (no repeated chorus, very long intros) fall back to the loudest early section.
- Mini window, global shortcuts and encrypted key storage need the new Windows installer; the "new app available" check needs the new APK.
- LRCLIB was tested with mocked responses (the build sandbox could not reach it); CORS from the web app should be confirmed on a real device.
- Android: no EQ or live analyser (direct audio). Pulse follows the beat grid there.
- Beat grid skipped for files over 14 MB; DJ falls back to crossfades for those.
- Picture-in-picture mini player only in Chromium browsers; Firefox/Safari show a notice.

15. Rules for anyone (or any AI) changing Aurora

- **User data must survive every update.** Never delete or reshape stored data destructively; add fields, migrate by merging, keep backups working (safety.js).
- **No paid AI or credits.** Features must work locally or with free, keyless services. The Anthropic key stays optional.
- **Confirm with Kailash before any release** (merging to main ships an over-the-air update to everyone).
- Bump the build (npm run bump) for every web change that should reach users; old builds never see an update otherwise.
- Never commit keys (*.pem, keystores). Never weaken the signature check, CSP, sandbox or folder allow-list.
- Keep the style: plain JS in the A namespace, one concern per file, compact code, comments that explain why, data-a actions, no frameworks or bundlers. New scripts go into index.html and the SHELL list in sw.js.
- Write user-facing text in plain, friendly language; keep phone layouts working (test at 390 px wide).

16. Glossary

Term	Meaning
OTA	Over-the-air update of the web code

Term	Meaning
Staged / live	Downloaded update waiting for restart / active update
Beat grid	Exact tempo, first beat, first downbeat and phrase map of a song
Phrase	8 bars (32 beats); DJs mix on phrase boundaries
Bass swap	Handing the low end from the outgoing to the incoming song mid-blend
Beat lock	Nudging playback speed so both songs' beats stay aligned
Journey	Planned energy path over a DJ set or a Flow
SAF	Android Storage Access Framework (folder permission)
PWA	Website that installs like an app
LRC	Lyrics with [mm:ss.xx] timestamps